COLLECTIONS

Coders.Eleven - JAVA

Coders.Eleven JAVA
Collections

#1 Wrapper

Allgemein

#2 Collections

Lists

Sets

Maps

#3 Iterator & Iterable

Motivation

Iterator

Iterable

#4 Weitere Collections

Queue

Stack



#1 Wrapper

Allgemein



Wrapper-Klassen

- Java bietet für jeden primitiven Datentyp sogenannte Wrapper-Klassen an
- Diese erfüllen drei Aufgaben:
  - Bieten statische Hilfsmethoden zur Konvertierung eines primitiven Datentyps in einen `String` (Formatierung) und von einem String in einen primitiven Datentyp (Parsen)
  - `Lists` und `Sets` (lernen wir gleich kennen) können nur Referenzen aufnehmen
    - Wrapper kapselt einen einfachen primitiven Wert in einem Objekt, sodass eine Referenz existiert
  - Auch bei `Generics` kommen Wrapper (ähnlich wie bei den Collections) zum Einsatz

Beispiel

- Erzeugen einiger Wrapper-Objekte:
  - Statische `valueOf()`-Methoden, denen ein primitiver Ausdruck oder ein String übergeben wird
  - Boxing: Aus einem primitiven Wert erzeugt der Compiler automatisch `valueOf()`-Methodenaufrufe, die das Wrapper-Objekt liefern

```
Integer  int1 = Integer.valueOf( s: "30" );   // valueOf()
Long     lng1 = Long.valueOf( l: 0xC0B0L );  // valueOf()
Boolean  bool = true;                        // Boxing
Integer  int3 = 42;                          // Boxing
```

```
30
49328
true
42
```

#2 Collections

Lists

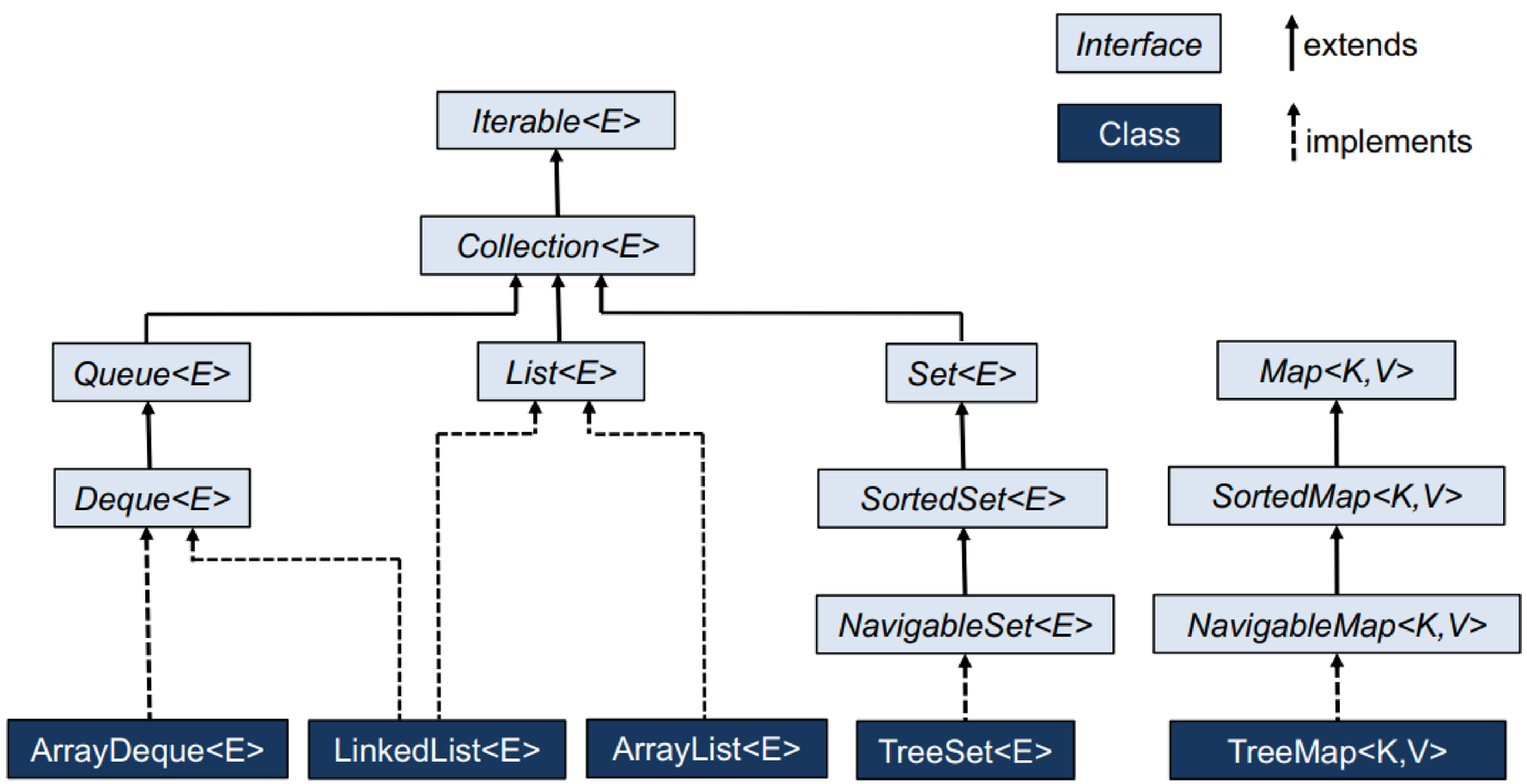
Sets

Maps



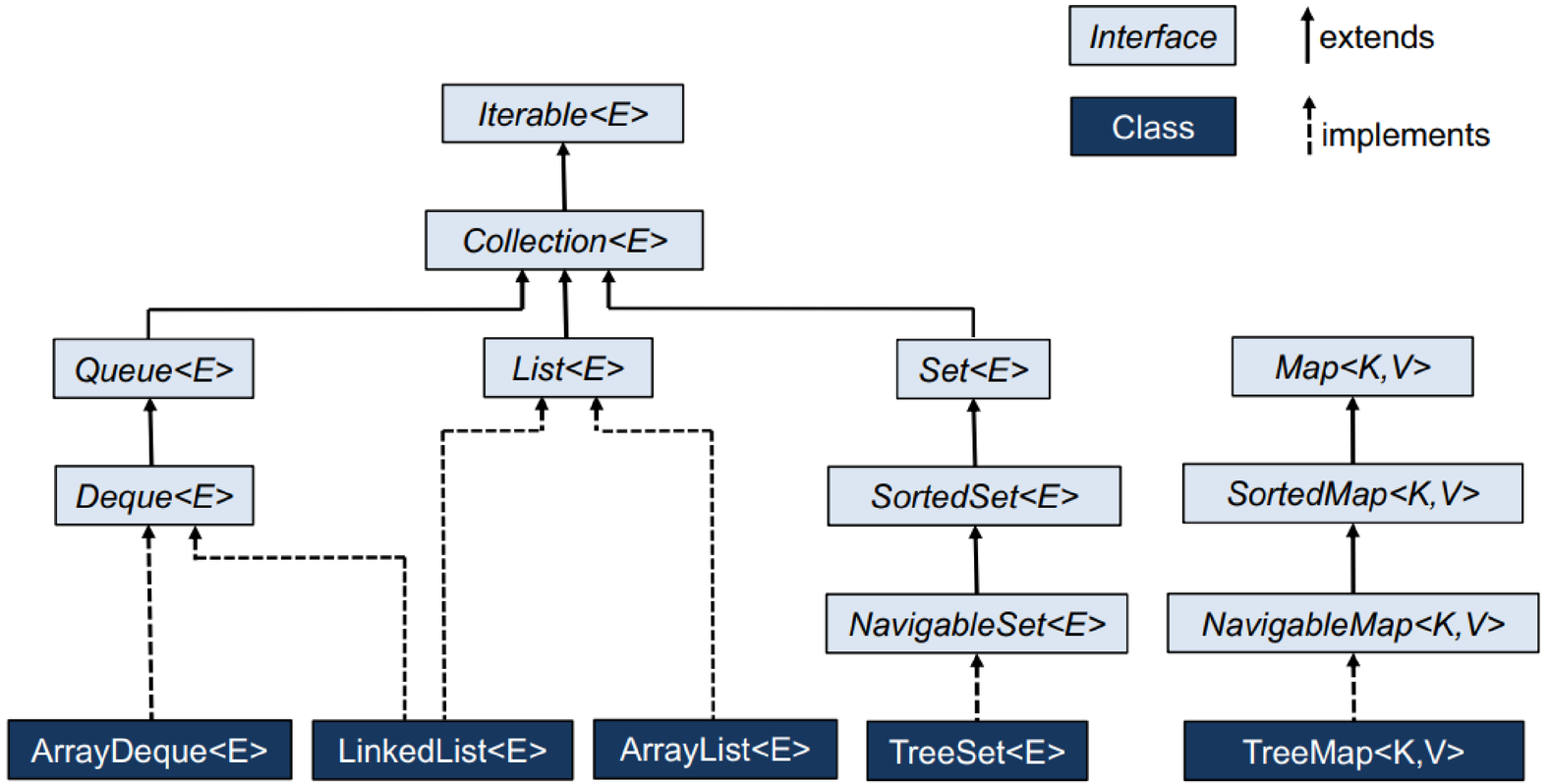
Allgemein

- Hierarchie der Collection & Map Typen
  - Nicht ganz vollständig (es fehlen zB abstrakte Klassen)



Collection Interface

- Alle Interfaces, welche unterhalb des Collection-Interfaces liegen, beinhalten die Methoden des Collection-Interfaces

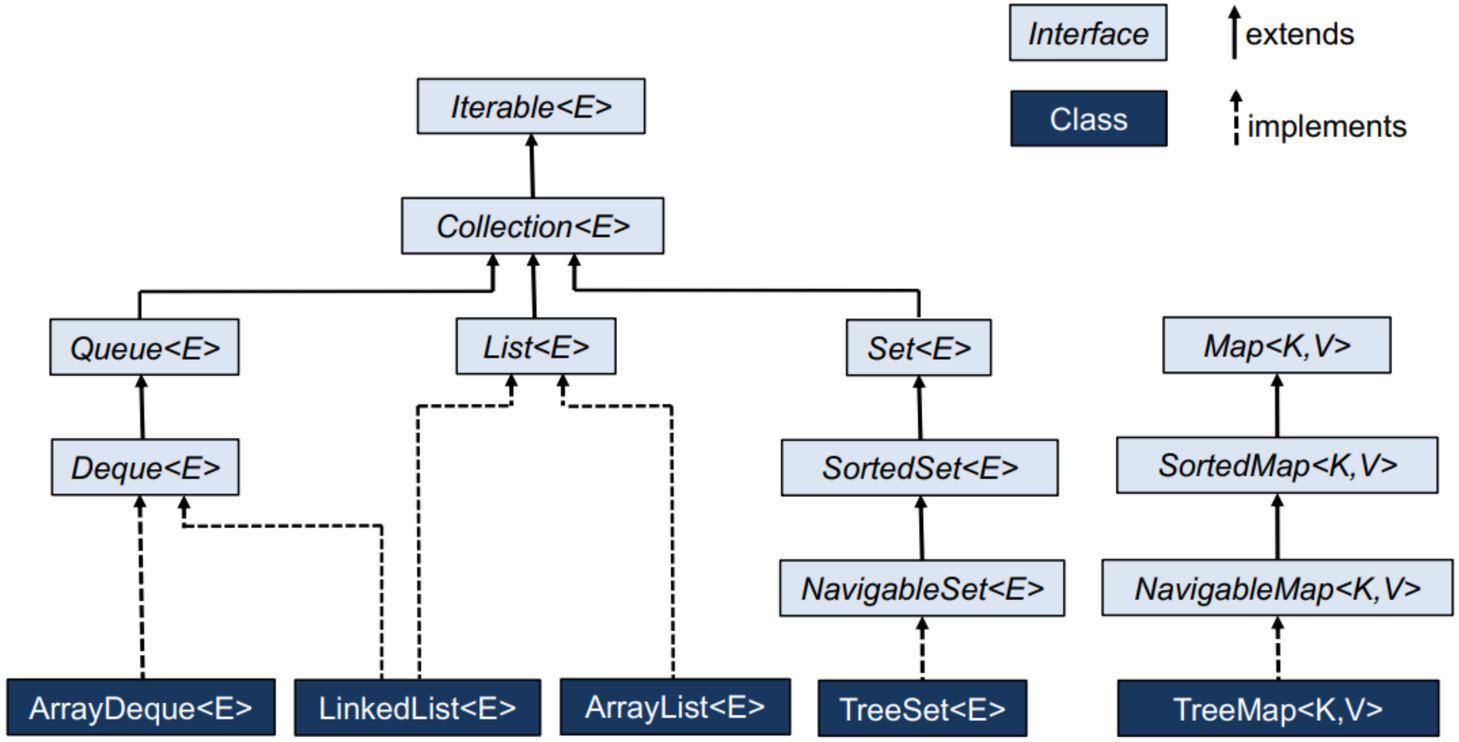


Collection Interface

- Einige Methoden:
  - `boolean add(Element e)` `// Fügt Element e der Collection hinzu`
  - `clear()` `// Entfernt alle Elemente aus der Collection`
  - `boolean contains(Element e)` `// Prüft, ob Element e in der Collection vorhanden ist`
  - `boolean equals(Element e)` `// Vergleicht Element e mit der Collection`
  - `Iterator<Object> iterator()` `// Iteriert über die Collection`
  - `boolean remove(Element e)` `// Entfernt Element e aus der Collection`
  - `E[] toArray()` `// Liefert ein Array, das alle Elemente der Collection enthält`
- Achtung: Das sind bei weitem nicht alle Methoden!
  - Für eine genauere Beschreibung siehe Javadoc

List Interface

- Schreibt das allgemeine Verhalten für Listen vor
- Viele Methoden kennen wir schon, weil List das Interface Collection erweitert
  - Hinzu kommen Methoden, die einen Bezug zur Position des Elements haben (Index)
  - Javadoc: List Interface



List Interface

- Einige Methoden:
  - `void add(int index, Element e)` `// Fügt Element e der Liste an Position index hinzu`
  - `Iterator<E> listIterator(int index)` `// Iteriert über die Liste, beginnend bei index`
  - `E remove(int index)` `// Entfernt das Element an der Stelle index aus der Liste`
  - `List<E> subList(int fromIndex, int toIndex)` `// Liefert eine Liste, beginnend bei fromIndex (inklusive) bis toIndex (exklusive)`
- Achtung: Das sind bei weitem nicht alle Methoden!
  - Für eine genauere Beschreibung siehe Javadoc

ArrayList<E>

- Realisierung einer Liste, die intern als Array verarbeitet wird
- Elemente werden lückenlos in einem Array gespeichert
  - Beim Einfügen bzw. Löschen müssen Elemente verschoben werden
- Ist das Array voll, muss ein größeres Array erzeugt, und die Elemente in dieses kopiert werden
- Um eine `ArrayList<E>` zu erstellen, gibt es drei Möglichkeiten:
  - `ArrayList()` `// Erzeugt eine leere ArrayList mit der Länge 10`
  - `ArrayList(Collection<E> c)` `// Erzeugt eine ArrayList, welche alle Elemente der Collection c enthält`
  - `ArrayList(int initialCapacity)` `// Erzeugt eine leere ArrayList mit der Länge initialCapacity`
- Für eine genauere Beschreibung aller Methoden siehe Javadoc

Arbeitsweise der ArrayList<E>

- Anzahl der Elemente in der ArrayList liefert die Methode `size()`
- Sollen aber mehr Elemente gespeichert werden, als eigentlich Platz ist, vergrößert sich die ArrayList automatisch (Resizing)
  - Neuer Speicherbereich muss reserviert werden und alle Elemente müssen in den neuen Speicherbereich kopiert werden
  - Wie groß ist das neue Array?
    - Alte Größe * 1.5
- ArrayList hat Vector abgelöst
  - Vector sollte nicht mehr verwendet werden (Legacy-Klasse)
  - Vector hat sich immer verdoppelt (führt schnell zu Problemen)

LinkedList<E>

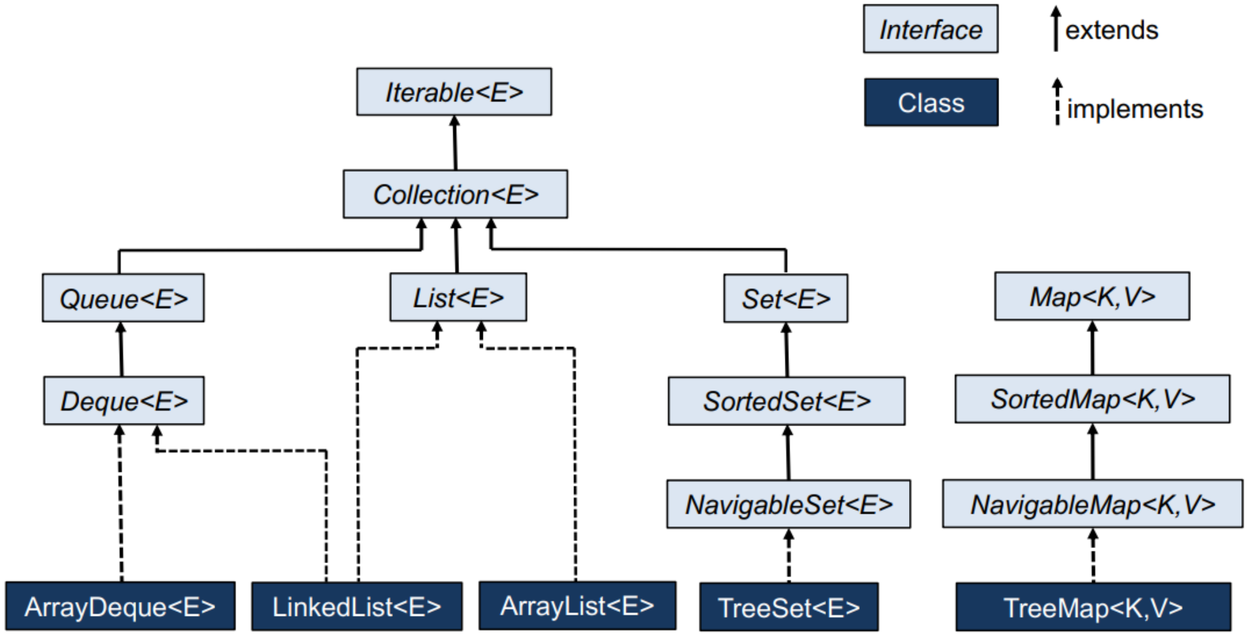
- Realisierung als doppelt verkettete Liste
- Jedes Element kennt seinen Vor- und Nachgänger
- Um eine `LinkedList<E>` zu erstellen, gibt es zwei Möglichkeiten:
  - `LinkedList()` `// Erzeugt eine leere LinkedList`
  - `LinkedList(Collection<E> c)` `// Erzeugt eine LinkedList, welche alle Elemente der Collection c enthält`
- Für eine genauere Beschreibung aller Methoden siehe Javadoc

ArrayList<E> vs. LinkedList<E>

Beschreibung	Operation	`ArrayList`	`LinkedList`
Ein Element abrufen	`get`	Schnell	Langsam
Element ändern	`set`	Schnell	Langsam
Element am Ende der Liste hinzufügen	`add`	Schnell	Schnell
Element an beliebiger Stelle einfügen	`add(index, element)`	Langsam	Schnell
Element entfernen	`remove`	Langsam	Schnell

Set Interface

- Schreibt das allgemeine Verhalten für Sets vor
- Viele Methoden kennen wir schon, weil Set das Interface Collection erweitert
  - Es kommen (fast) keine neuen Methoden hinzu, jedoch werden die vorhandenen Methoden verschärft
  - Javadoc: Set Interface



Set Interface

- Einige Methoden:
  - `boolean add(Element e)` `// Fügt e dem Set hinzu, falls das Element noch nicht im Set vorhanden ist`
  - `boolean addAll(Collection<E> c)` `// Fügt c dem Set hinzu, falls die Elemente noch nicht im Set vorhanden sind`

- Alle anderen Methoden sind denen aus dem Collection Interface sehr ähnlich
- Achtung: Das sind bei weitem nicht alle Methoden!
  - Für eine genauere Beschreibung siehe Javadoc

SortedSet<E> & NavigableSet<E>

- `SortedSet`:
  - Menge, deren Elemente sortiert sind
  - Elemente können auf zwei Arten sortiert sein:
    - `compareTo` (natürliche Ordnung)
    - `Comparator`-Objekt (wird beim Konstruktoraufruf übergeben)
  - Für eine genauere Beschreibung siehe Javadoc
- NavigableSet:
  - Es gibt (im Unterschied zu `SortedSet`) Navigationsmethoden, die zu einem Element das nächstkleinere bzw. größere Element zurückliefern
    - Es geht auch kleiner-gleich bzw. größer-gleich
  - Für eine genauere Beschreibung siehe Javadoc

TreeSet<E>

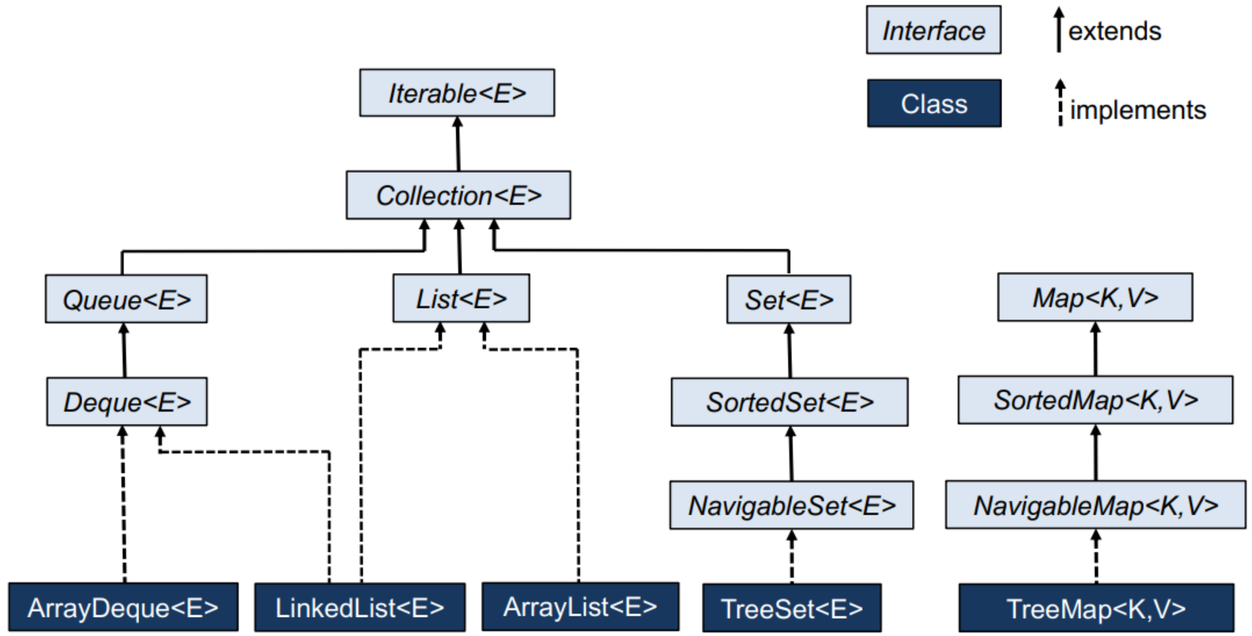
- `TreeSet` ist eine Implementierung als balancierter Binärbaum (genauer: Rot-Schwarz-Baum)
- Wichtige Methoden wie add, remove und contains benötigen nur eine Laufzeit von O(log n)
- Es gibt vier Möglichkeiten ein TreeSet zu erzeugen:
  - `TreeSet()` `// Erzeugt ein neues, leeres TreeSet, sortiert nach der natürlichen Ordnung`
  - `TreeSet(Collection<E> c)` `// Erzeugt ein neues TreeSet aus c, sortiert nach der natürlichen Ordnung`
  - `TreeSet(Comparator<E> comp)` `// Erzeugt ein neues, leeres TreeSet, sortiert nach comp`
  - `TreeSet(SortedSet<E> s)` `// Erzeugt ein neues TreeSet mit den Elementen und der Sortierung von s`
  - Für eine genauere Beschreibung siehe Javadoc

Map

- Maps sind Mengen von Key-Value-Pairs (Schlüssel-Wert-Paare), wobei ein Key nicht mehrfach vorkommen darf
- Eine Map bildet einen Key auf einen Value ab
  - Deshalb der Name Map (= Abbildung)
- Beispiel: Telefonbuch
  - Key: Telefonnummer
  - Value: Nachname

Map Interface

- Schreibt das allgemeine Verhalten für Maps vor
- Map<K, V> ist ein generischer Typ
  - K = Key, V = Value
- Maps selbst sind nicht iterierbar
  - Javadoc: Map Interface



Map Interface

- Einige Methoden:
  - `V get(Object key)` `// Liefert den Value zum angegebenen Key`
  - `Set<K> keySet()` `// Liefert ein Set, welches alle Keys enthält`
  - `V put(K key, V value)` `// Fügt den Key und den Value zur Map hinzu`
  - `V remove(Object key)` `// Entfernt den Value zum angegebenen Key`
  - `Collection<V> values()` `// Liefert eine Collection, welche alle Values der Map enthält`

- Achtung: Das sind bei weitem nicht alle Methoden!
  - Für eine genauere Beschreibung siehe Javadoc

Sorted-, Navigable-, Tree- & HashMap

- SortedMap, NavigableMap und TreeMap sind analog zu SortedSet, NavigableSet und TreeSet
  - Die Elemente sind nach ihrem Key sortiert
  - TreeMap bietet einen Konstruktor an, dem ein Vergleichsobjekt für die Festlegung der Reihenfolge der Keys übergeben werden kann
- HashMap
  - Ist eine Klasse und kein Interface
  - Kann gleiche Values beinhalten
  - Es ist ein null-Key erlaubt, welchem ein Value zugewiesen werden kann
- Für genauere Beschreibungen siehe: SortedMap, NavigableMap, TreeMap, HashMap

#3 Iterator & Iterable

Motivation

Iterator

Iterable



Motivation

- Nicht nur einzelne Daten, sondern auch Sammlungen von Daten spielen eine Rolle in Programmen
- Diese Sammlungen haben meistens Methoden, die Daten hinzufügen, entfernen und prüfen, ob bestimmte Elemente vorhanden sind
- Dieses Verhalten wird oftmals durch Interfaces gelöst
- Zwei wichtige Interfaces haben wir noch nicht besprochen:
  - `Iterator`
    - Bietet eine Möglichkeit, Schritt für Schritt durch Sammlungen zu laufen
  - `Iterable`
    - Liefert so einen Iterator, ist also ein Iterator-Produzent

Iterator Interface

- Verfügt über zwei zentrale Methoden:
  - `hasNext()`: Liefert `true`, falls die Iteration noch weitere Elemente bietet
  - `next()`: Liefert das nächste Element in der Aufzählung und setzt die Position weiter

    Es gibt eine `NoSuchElementException`, wenn keine weiteren Elemente vorhanden sind
- Beispiel, um eine `ArrayList` zu durchlaufen:

```
ArrayList<String> list = new ArrayList<>();
Iterator<String> iter = list.iterator();
while (iter.hasNext()){
    String element = iter.next();
    System.out.println("Aktuelles Element: " + element);
}
```

Richtung des Iterators

- Iterator arbeitet nur vorwärts
- Die `next()`-Methode geht immer zum nächsten Element
  - Es ist nicht möglich, ein Element nochmals auszulesen oder hin- und herzuspringen
- Grundsätzlich entscheidet die Implementierung der Datenstruktur (und des Iterators), in welcher Reihenfolge die Elemente herausgegeben werden
  - Manchmal ist es möglich, auch von hinten zu iterieren

Elemente über Iterator entfernen

- Die `next()`-Methode ist nur zum Lesen und verändert die Datenstruktur nicht
- Zu den zwei bisher besprochenen Methoden gibt es noch eine dritte:
  - `remove()`
    - Löscht das zuletzt von `next()` gelieferte Objekt aus der darunterliegenden Datenstruktur

      Muss nicht zwingend angeboten werden und löst eine

      `UnsupportedOperationException(“remove“)` aus

Iterable Interface

- Eine Methode `iterator()`, die einen Iterator liefert, ist häufig anzutreffen
- Grund: Die Klassen mit `iterator()`-Methode implementieren das Interface `Iterable`, welches eine Methode `iterator()` definiert
- `Iterator<T> iterator()`
  - Liefert einen Iterator, der über alle Elemente vom Typ T iteriert
- Wegen beiden Interfaces kann zum Iterieren auch die `for-each`-Schleife statt der `while`-Schleife verwendet werden:

```
Iterator<String> iter = list.iterator();
while (iter.hasNext()){
    String element = iter.next();
    System.out.println("Aktuelles Element: " + element);
}
```

```
for (String element : list) {
    System.out.println("Aktuelles Element: " + element);
}
```

#4 Weitere Collections

Queue

Stack



Queues

- Interface Queue definiert die typischen Operationen für (Warte-) Schlangen
- Arbeitet nach dem FIFO-Prinzip
  - First In First Out, d.h. Ablegen geschieht von hinten, das Entnehmen von vorne

	Methode liefert `false` bzw. `null` bei Misserfolg	Methode wirft Exception bei Misserfolg
Einfügen am Ende	`boolean offer(E elem)`	`boolean add(E elem)`
Löscht und liefert das vordere Element	`E poll()`	`E remove()`
Liefert das vordere Element	`E peek()`	`E element()`

Deques

- Interface Deque ist eine Schlange, bei der an beiden Enden eingefügt und gelöscht werden kann (double-ended queue)

Methoden für vorderes Ende	Methode liefert `false` bzw. `null` bei Misserfolg	Methode wirft Exception bei Misserfolg
Einfügen	`boolean offerFirst(E elem)`	`void addFirst(E elem)`
Löschen und zurückliefern	`E pollFirst()`	`E removeFirst()`
Zurückliefern	`E peekFirst()`	`E getFirst()`

Methoden für hinteres Ende	Methode liefert `false` bzw. `null` bei Misserfolg	Methode wirft Exception bei Misserfolg
Einfügen	`boolean offerLast(E elem)`	`void addLast(E elem)`
Löschen und zurückliefern	`E pollLast()`	`E removeLast()`
Zurückliefern	`E peekLast()`	`E getLast()`

Deque als Stack

- Deque<E> kann auch als Stack verwendet werden
- Achtung: Es gibt in der Java-API auch eine Klasse Stack<E>, welche jedoch veraltet ist
- Stack arbeitet nach dem LIFO-Prinzip
  - Last In First Out, d.h. Ablegen und Entnehmen von Objekten geschieht von oben

Stack-Methode	Äquivalente Deque-Methode	Exception bzw. spezieller Rückgabewert
void push(E elem)	`void addFirst(E elem)`	Exception
E pop()	`E removeFirst()`	Exception und Rückgabe
E peek()	`E peekFirst()`	Rückgabe (kann null sein)